AI-Based Hospital Appointment Scheduling System

Source Code Listing (hospital_system.c), terminal application in C, no GUI

Devvrat Tripathi | Roll No. 250133150078 | B.Tech AI, Semester 3, Section B
Course: Data Structures and Algorithms - II (BCSE0401) | Faculty: Mr Vikas Tripathi

```c
/*
    AI-Based Hospital Appointment Scheduling System
    DSA-II Individual Assignment (BCSE0401), terminal app, no GUI.

    Compile : gcc hospital_system.c -o hospital_system
    Run     : ./hospital_system      (Windows: hospital_system.exe)

    Unit 1 : BST patient registry + max-heap appointment queue + heap sort
    Unit 2 : Dijkstra shortest path on the hospital campus graph
    Unit 3 : 0/1 Knapsack DP for operation theatre slot allocation
    Unit 4 : Sum of subsets backtracking for exact free-slot packing
    Unit 5 : B+ tree indexing is discussed in the report.
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_PATIENTS 100
#define N_NODES 10
#define INF 999999
#define MAX_SURG 8
#define MAX_CAP 480

typedef struct {
    int id;
    char name[30];
    char disease[30];
    int severity;
} Patient;

Patient patient_list[MAX_PATIENTS];
int patient_count = 0;

/* ================= Unit 1: BST patient registry ================= */
typedef struct BSTNode {
    Patient p;
    struct BSTNode *left, *right;
} BSTNode;
BSTNode *root = NULL;

BSTNode *bst_insert(BSTNode *node, Patient p) {
    if (node == NULL) {
        BSTNode *n = (BSTNode *)malloc(sizeof(BSTNode));
        n->p = p;
        n->left = NULL;
        n->right = NULL;
        return n;
    }
    if (p.id < node->p.id)
        node->left = bst_insert(node->left, p);
    else if (p.id > node->p.id)
        node->right = bst_insert(node->right, p);
    return node;
}

Patient *bst_search(BSTNode *node, int id) {
    while (node != NULL) {
        if (id == node->p.id)
            return &node->p;
        if (id < node->p.id)
            node = node->left;
        else
            node = node->right;
    }
    return NULL;
}

```

```c
void bst_inorder(BSTNode *node) {
    if (node == NULL)
        return;
    bst_inorder(node->left);
    printf("   ID %-4d %-15s %-13s severity %d\n", node->p.id, node->p.name, node->p.disease, node->p.severity);
    bst_inorder(node->right);
}

/* ================= Unit 1: max-heap priority queue ================= */
void heap_push(Patient h[], int *size, Patient p) {
    int i = *size;
    h[i] = p;
    (*size)++;
    while (i > 0) {
        int parent = (i - 1) / 2;
        Patient tmp;
        if (h[i].severity <= h[parent].severity)
            break;
        tmp = h[i];
        h[i] = h[parent];
        h[parent] = tmp;
        i = parent;
    }
}

Patient heap_pop(Patient h[], int *size) {
    Patient top = h[0];
    int i = 0;
    (*size)--;
    h[0] = h[*size];
    while (1) {
        int l = 2 * i + 1;
        int r = 2 * i + 2;
        int big = i;
        Patient tmp;
        if (l < *size && h[l].severity > h[big].severity)
            big = l;
        if (r < *size && h[r].severity > h[big].severity)
            big = r;
        if (big == i)
            break;
        tmp = h[i];
        h[i] = h[big];
        h[big] = tmp;
        i = big;
    }
    return top;
}

/* ================= Unit 2: hospital campus graph + Dijkstra ================= */
char *nodes[N_NODES] = {"Reception", "OPD", "ER", "ICU", "OT",
                        "Pharmacy", "Pathology", "Radiology", "GeneralWard", "Cafeteria"};
int graph[N_NODES][N_NODES];

void init_graph() {
    int edges[][3] = {
        {0,1,3}, {0,2,5}, {0,5,4}, {1,6,6}, {1,7,7},
        {2,3,4}, {2,4,8}, {3,4,3}, {3,8,5}, {4,8,4},
        {6,5,5}, {7,5,6}, {8,9,2}, {5,9,3}
    };
    int m = sizeof(edges) / sizeof(edges[0]);
    int i, j;
    for (i = 0; i < N_NODES; i++)
        for (j = 0; j < N_NODES; j++)
            graph[i][j] = (i == j) ? 0 : INF;
    for (i = 0; i < m; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        int w = edges[i][2];
        graph[u][v] = w;
        graph[v][u] = w;
    }
}

void dijkstra(int src, int dst) {
    int dist[N_NODES], prev[N_NODES], visited[N_NODES], path[N_NODES];
    int i, count, v, len = 0;
    for (i = 0; i < N_NODES; i++) {
        dist[i] = INF;
```

```
        prev[i] = -1;
        visited[i] = 0;
    }
    dist[src] = 0;
    for (count = 0; count < N_NODES; count++) {
        int u = -1, best = INF;
        for (i = 0; i < N_NODES; i++) {
            if (visited[i] == 0 && dist[i] < best) {
                best = dist[i];
                u = i;
            }
        }
        if (u == -1)
            break;
        visited[u] = 1;
        for (v = 0; v < N_NODES; v++) {
            if (visited[v] == 0 && graph[u][v] != INF) {
                int nd = dist[u] + graph[u][v];
                if (nd < dist[v]) {
                    dist[v] = nd;
                    prev[v] = u;
                }
            }
        }
    }
    if (dist[dst] == INF) {
        printf("   No path found.\n");
        return;
    }
    for (v = dst; v != -1; v = prev[v])
        path[len++] = v;
    printf("   Shortest path %s -> %s = %d minutes\n", nodes[src], nodes[dst], dist[dst]);
    printf("   Route: ");
    for (i = len - 1; i >= 0; i--)
        printf("%s%s", nodes[path[i]], (i == 0) ? "\n" : " -> ");
}

/* ================= Unit 3: 0/1 knapsack DP for OT slots ================= */
typedef struct {
    char name[25];
    int duration;
    int priority;
} Surgery;

void knapsack_ot() {
    Surgery s[MAX_SURG] = {
        {"Appendectomy", 90, 8}, {"Knee replacement", 180, 10},
        {"Gallbladder removal", 120, 9}, {"Cataract", 45, 5},
        {"Hernia repair", 75, 7}, {"Angioplasty", 150, 10},
        {"Biopsy", 30, 4}, {"Tonsillectomy", 60, 6}
    };
    int dp[MAX_SURG + 1][MAX_CAP + 1];
    int i, c;
    memset(dp, 0, sizeof(dp));
    for (i = 1; i <= MAX_SURG; i++) {
        for (c = 0; c <= MAX_CAP; c++) {
            dp[i][c] = dp[i - 1][c];
            if (s[i - 1].duration <= c) {
                int take = dp[i - 1][c - s[i - 1].duration] + s[i - 1].priority;
                if (take > dp[i][c])
                    dp[i][c] = take;
            }
        }
    }
    printf("   OT capacity = 480 min (8 hours)\n");
    printf("   Max total priority (DP) = %d\n", dp[MAX_SURG][MAX_CAP]);
    printf("   Selected surgeries:\n");
    c = MAX_CAP;
    for (i = MAX_SURG; i >= 1; i--) {
        if (dp[i][c] != dp[i - 1][c]) {
            printf("   - %-20s %4d min, priority %d\n", s[i - 1].name, s[i - 1].duration, s[i - 1].priority);
            c = c - s[i - 1].duration;
        }
    }
}

/* ================= Unit 4: sum of subsets backtracking ================= */
int found;

```

```c
void subset_search(int a[], int n, int idx, int sum, int target, int chosen[], int len) {
    int i;
    if (found == 1)
        return;
    if (sum == target) {
        printf("   Subset found: {");
        for (i = 0; i < len; i++) {
            if (i > 0)
                printf(", ");
            printf("%d", chosen[i]);
        }
        printf("} = %d min\n", target);
        found = 1;
        return;
    }
    if (idx == n || sum > target)
        return;
    chosen[len] = a[idx];
    subset_search(a, n, idx + 1, sum + a[idx], target, chosen, len + 1);
    subset_search(a, n, idx + 1, sum, target, chosen, len);
}

/* ================= sample data and menu ================= */
void load_sample_data() {
    Patient list[] = {
        {104, "Aman Verma", "Fever", 3}, {101, "Priya Singh", "Fracture", 7},
        {107, "Rohit Das", "Migraine", 4}, {103, "Sara Khan", "Asthma", 6},
        {109, "Kunal Mehta", "Diabetes", 5}, {102, "Neha Gupta", "Cough", 2},
        {105, "Arjun Rao", "Cardiac", 9}
    };
    int n = sizeof(list) / sizeof(list[0]);
    int i;
    for (i = 0; i < n; i++) {
        root = bst_insert(root, list[i]);
        patient_list[patient_count] = list[i];
        patient_count++;
    }
}

void add_patient() {
    Patient p;
    printf("   Enter patient ID, name, disease, severity (1-10):\n   ");
    scanf("%d %s %s %d", &p.id, p.name, p.disease, &p.severity);
    if (bst_search(root, p.id) != NULL) {
        printf("   Patient ID already exists.\n");
        return;
    }
    root = bst_insert(root, p);
    patient_list[patient_count] = p;
    patient_count++;
    printf("   Patient added to registry and appointment queue.\n");
}

void show_heap(int sort_only) {
    Patient h[MAX_PATIENTS];
    int size = 0, i;
    for (i = 0; i < patient_count; i++)
        heap_push(h, &size, patient_list[i]);
    if (sort_only == 1)
        printf("   Heap sort by severity, O(n log n):\n");
    else
        printf("   Appointment queue, most urgent first:\n");
    while (size > 0) {
        Patient p = heap_pop(h, &size);
        if (sort_only == 1)
            printf("   ID %-4d %-15s severity %d\n", p.id, p.name, p.severity);
        else
            printf("   ID %-4d %-15s %-13s severity %d\n", p.id, p.name, p.disease, p.severity);
    }
}

void dijkstra_menu() {
    int src, dst, i;
    printf("   Departments:\n");
    for (i = 0; i < N_NODES; i++)
        printf("   %d = %s\n", i, nodes[i]);
    printf("   Enter source and destination indices: ");
    scanf("%d %d", &src, &dst);
    if (src < 0 || src >= N_NODES || dst < 0 || dst >= N_NODES) {
```

```
        printf("   Invalid department index.\n");
        return;
    }
    dijkstra(src, dst);
}

void sum_of_subsets_demo() {
    int durations[] = {30, 45, 60, 20, 35};
    int chosen[5];
    found = 0;
    printf("   Free slot = 80 min, leftover requests = {30, 45, 60, 20, 35}\n");
    subset_search(durations, 5, 0, 0, 80, chosen, 0);
    if (found == 0)
        printf("   No exact subset found.\n");
}

int main() {
    int choice;
    init_graph();
    load_sample_data();
    do {
        printf("\n===== AI-Based Hospital Appointment Scheduling System =====\n"
               "1. Show all patients (BST inorder)\n"
               "2. Search patient by ID\n"
               "3. Add new patient\n"
               "4. Show appointment queue (heap)\n"
               "5. Heap sort patients by severity\n"
               "6. Shortest path between departments (Dijkstra)\n"
               "7. OT slot allocation (0/1 knapsack DP)\n"
               "8. Pack free slot exactly (sum of subsets)\n"
               "9. Exit\nChoice: ");
        scanf("%d", &choice);
        switch (choice) {
        case 1:
            printf("   Patients in BST, inorder (sorted by ID):\n");
            bst_inorder(root);
            break;
        case 2: {
            int id;
            printf("   Enter patient ID: ");
            scanf("%d", &id);
            Patient *p = bst_search(root, id);
            if (p != NULL)
                printf("   Found: ID %d, %s, %s, severity %d\n", p->id, p->name, p->disease, p->severity);
            else
                printf("   Patient not found.\n");
            break;
        }
        case 3: add_patient(); break;
        case 4: show_heap(0); break;
        case 5: show_heap(1); break;
        case 6: dijkstra_menu(); break;
        case 7: knapsack_ot(); break;
        case 8: sum_of_subsets_demo(); break;
        case 9: printf("   Exiting.\n"); break;
        default: printf("   Invalid choice, try again.\n");
        }
    } while (choice != 9);
    return 0;
}
```

End of source code (364 lines). Compile with: gcc hospital_system.c -o hospital_system